Solution Report: Mission 3 Author: Syed Hashir Sohail

Team: Squad Sutskever
Team members: Syed Hashir Sohail, Yasan Malik, Usman Tariq Qureshi, Sufyan Khalid
Prepared: 26 October 2026 ## 1. Overview and evidence Mission 3 investigates multi-head attention, rotary positional embeddings, grouped-query attention, KV-cache storage, and memory safety. This report describes current source files and previously recorded outputs. No experiments were rerun for this report. Recorded measurements, analytical estimates, and unverified implementation features are distinguished in the discussion. Codex assisted with implementation, verification, and documentation, as disclosed in the project materials, and with preparing this report. Notebook outputs and the benchmark report document earlier CPU runs. The missing attention and GQA source modules must be restored before those components can be reproduced from the current mission folder. ## 2. Mission 3: Transformer components and memory

2.1 Multi-head self-attention

The retained ps03_solution.ipynb describes projection of an input of shape [B, N, D] into queries, keys, and values, splitting across H heads, and computing Attention(Q,K,V) = softmax(QK^T / sqrt(d_k) + mask)V. Head outputs are concatenated and projected back to the model dimension. Causal masking excludes future keys; padding masks exclude invalid positions.

The notebook records CPU execution with PyTorch 2.14.1+cpu and CUDA unavailable. Its example maps [2, 8, 64] to [2, 8, 64], with attention weights [2, 4, 8, 8]. At head dimension 64, recorded unscaled dot-product variance is 64.1756 and scaled variance is 1.0027, consistent with the purpose of the sqrt(d_k) denominator.

Recorded checks pass for an independent PyTorch SDPA reference, row sums, causal isolation, padding and fully masked rows, finite gradients, input validation, and finite FP16/BF16 outputs. The strictly upper triangular causal weights contain zero nonzero elements. A recorded compatibility check matches MHA against GQA when query and KV head counts are equal, within 1e-6 tolerance.

The source module 01_MultiHead_Attention_Core_starter.py is currently absent. The notebook output documents an earlier run; rerunning it requires restoring that module and the GQA dependency.

2.2 Rotary positional embeddings

The current implementation is mission-03/02_RoPE_Positional_Engine.py. RoPE groups adjacent feature coordinates into pairs and applies a position-dependent planar rotation, represented using complex multiplication. The rotation preserves pairwise magnitude. With consistent frequencies, query-key dot products depend on relative position rather than a shared absolute shift.

The script includes relative-invariance verification, shape and norm demonstrations, a RoPE-versus-ALiBi attention heatmap, and an NTK-scaling demonstration. These are implementation features; no new plots or successful execution are claimed here. They illustrate positional behaviour and do not demonstrate trained-model long-context quality. Integration of rotated keys with the GQA cache remains unverified in the benchmark report: each new key must be rotated once at its absolute position before storage.

2.3 GQA, MQA, and KV-cache design

Grouped-query attention shares key/value heads across multiple query heads. MHA uses one KV head per query head, GQA uses an intermediate count, and MQA uses one KV head. A stateful cache retains prior keys and values for token-by-token or chunked causal decoding.

Cache storage in bytes is 2 * B * H_KV * N * d_head * bytes_per_element, where the factor two represents keys and values. At equal batch, length, head dimension, and dtype, moving from 32 KV heads to eight reduces storage by 75%; moving to one reduces it by 96.875%. Storage savings alone do not establish better latency or equal language-model quality.

2.4 Recorded CPU benchmark

The existing benchmark report describes a 22 October 2026 macOS ARM CPU run with Python 3.14.3, PyTorch 2.14.1, four CPU threads, batch one, 32 query heads, head dimension 128, float16, and seed 42. Each configuration uses two warm-ups and five timed repeats. The timed operation decodes one new token against a synthetic projected prefix; prefix construction is excluded.

Configuration	Context	Cache MiB	Median CPU ms
MHA (32:32)	1,024	16.00	1.786
GQA (32:8)	1,024	4.00	1.995
MQA (32:1)	1,024	0.50	1.343
MHA (32:32)	4,096	64.00	2.833
GQA (32:8)	4,096	16.00	5.731
MQA (32:1)	4,096	2.00	2.566
MHA (32:32)	16,384	256.00	7.242
GQA (32:8)	16,384	64.00	21.698
MQA (32:1)	16,384	8.00	6.531
MHA (32:32)	32,768	512.00	13.248
GQA (32:8)	32,768	128.00	49.188
MQA (32:1)	32,768	16.00	11.880

Cache figures are calculated tensor capacities; latency figures are reported CPU measurements. They are not GPU memory readings, full-model generation timings, or general hardware rankings. In these runs GQA reduces cache size but is slower than MHA at every tested context length. Explicit grouped operations and backend behaviour can outweigh storage savings.

The report records passing independent-reference, token/chunk cache equivalence, and causal-isolation checks for Q:KV ratios 4:4, 4:2, and 4:1, plus cache append, storage, reset, bounds, and dtype checks. The corresponding 03_GQA_and_KVCache_Benchmark_starter.py and machine-readable result file are absent from the current working tree, limiting immediate reproducibility.

2.5 Memory hierarchy and safety audit

The retained memory audit separates full attention intermediates from persistent KV storage. Standard attention materialises quadratic score/probability data. FlashAttention tiles computation to reduce movement between high-bandwidth memory and SRAM and avoids storing the complete quadratic intermediates. It does not eliminate quadratic full-sequence arithmetic or persistent KV-cache growth.

For the audit’s model, standard attention IO is Theta(N*d + N^2); the tiled bound is O(N*d + N^2*d^2/M), where M is SRAM capacity in scalar elements and the bound depends on the stated cache regime. At fixed d and M, the second term remains quadratic. Describing this as unconditionally linear IO would be inaccurate.

Under the audit’s simplified capacity assumptions, a 65,536-token session needs 32 GiB for

MHA, 8 GiB for GQA, or 1 GiB for MQA. A dedicated 64 GiB KV budget fits two, eight, or 64 such sessions respectively; the next session exceeds that budget. These are analytical examples, not observed CUDA out-of-memory events. Real capacity depends on weights, activations, fragmentation, temporary buffers, and runtime headroom.

The audit recommends admission budgeting, tenant quotas, context and concurrency limits, monitoring, and graceful overload handling. PagedAttention addresses allocation and fragmentation; GQA and latent-cache approaches reduce bytes per token. These techniques complement enforceable resource limits rather than guaranteeing unlimited capacity. ## 3. Team allocation and review status The allocation targets 25% of mission responsibilities per person. It is a plan, not measured historical contribution or completed sign-off. Syed Hashir Sohail: attention core and shared notebook coordination.

Yasan Malik: memory hierarchy and safety audit.

Usman Tariq Qureshi: RoPE positional engine.

Sufyan Khalid: GQA/cache implementation and benchmark. Current team reviews remain pending in TEAM_ALLOCATION.md. Historical completion checkboxes do not establish current sign-off. ## 4. Reproduction guide Run the following commands from the repository root after restoring any missing dependencies: python mission-03/02_RoPE_Positional_Engine.py

```
python mission-03/01_MultiHead_Attention_Core_starter.py
python mission-03/03_GQA_and_KVCache_Benchmark_starter.py --verify-only  Use a Python environment with PyTorch and NumPy, plus Matplotlib for plotting. Restore the attention and GQA source modules before running those commands or the full notebook. ps03_solution.ipynb supports its documented project-root or mission-directory layout. Previous output does not establish that a fresh run succeeded.  ## 5. Local evidence sources  - ps03_solution.ipynb: recorded attention checks and compatibility output.
```

- 02_RoPE_Positional_Engine.py: rotary embedding implementation and demonstrations.
- results/gqa_kvcache_report.md: benchmark method, results, and limitations.
- 04_Memory_Hierarchy_and_Safety_Audit_template.md: analytical IO, capacity, and safety assessment.
- TEAM_ALLOCATION.md: planned responsibilities and pending reviews. The PDF is generated from this Markdown source.